Mini-Project (ML for Time Series) - MVA 2025/2026: Scalable Time Series Shapelet Discovery with FastShapelet

Theo BASSERAS theobasseras@gmail.com
Yani Hammache yahmce@gmail.com

December 14, 2025

1 Introduction and contributions

Time Series (TS) data classification is a fundamental challenge in diverse fields like finance, medicine, and human activity recognition. While classic methods like Nearest Neighbor with Dynamic Time Warping (DTW) provide high accuracy, they suffer from high computational cost and a critical lack of interpretability required for real-time and expert systems.

The Time Series Shapelet concept addresses these limitations. A shapelet is defined as a characteristic short pattern or subsequence whose presence is highly predictive of a specific class. Identifying a small, discriminative set of shapelets transforms the complex classification task into a faster, feature-based one, providing an easily interpretable model that generalizes the nearest neighbor approach.

However, the discovery of optimal shapelets is computationally prohibitive, involving a search space that leads to the intractable Brute Force complexity of $O(n^2m^4)$. This work details the scalable FastShapelet algorithm, which overcomes this issue. FastShapelet utilizes Symbolic Aggregate approXimation (SAX) and Random Projection hashing to efficiently filter the vast candidate space, drastically reducing the required number of expensive distance computations and making shapelet discovery feasible for large-scale data.

Our contributions and work distribution

- Repartition of work: Theo focused on Introduction, the definitions of the model and the algorithms different from the FastShapelet to compare with; Yani focused on the fast shapelet implementation and experimentation/data analysis.
- Use of available source code: We developed our Deep Learning model from scratch. For the *FastShapelet* algorithm, the core implementation was written by us. For the comparative methods, we transformed the barycenter computation code from the reference article [1] into Python. The original implementation can be found at: `https://github.com/g0bel1n/fast_shapelets`. This transformation contributed to a partial code reuse percentage.

2 Method

The goal of a shapelet discovery algorithm is to find the single subsequence s that, when used as a decision boundary, best separates the dataset D into two highly pure (single-class) subsets.

This section summarizes the necessary mathematical definitions and the algorithms for both the intractable Brute Force approach and the scalable FastShapelet.

2.1 Shapelet Optimality Definitions

Definition of a Split $\langle s, d \rangle$: A split is defined by a candidate shapelet s (a subsequence) and a distance threshold d. For a time series T, the distance $\text{dist}(s, T)$ is the minimum (normalized) Euclidean distance between s and any of its same-length subsequences in T. This split divides the dataset D into two subsets:

- $\mathbf{D_L}$ (Left Dataset): Time series where $\text{dist}(s, T) \leq d$. (Close to the shapelet).
- $\mathbf{D_R}$ (Right Dataset): Time series where $\text{dist}(s, T) > d$. (Far from the shapelet).

Definition of Information Gain I: The quality of a split is measured using Information Gain, based on the concept of entropy E (disorder/impurity). A good split maximizes the reduction in entropy.

$$I(s, d) = E(D) - \frac{n_L}{n} E(D_L) - \frac{n_R}{n} E(D_R)$$

where

$$E(D) = \sum_i p_i \log(p_i) \qquad p_i = \frac{n_i}{n}$$

, n_L, n_R, and n are the number of objects in D_L, D_R, and D respectively. Maximizing I results in the purest possible sub-datasets.

Definition of the Separation gap: An alternative metric, often used to resolve ties in Information Gain, measures the separation between the average distances in the two resulting sets:

$$\text{gap}(s, d) = \frac{1}{|D_R|} \sum_{T \in D_R} \text{dist}(s, T) - \frac{1}{|D_L|} \sum_{T \in D_L} \text{dist}(s, T)$$

A larger gap indicates a more distinct separation, suggesting a better shapelet.

2.2 Brute Force and Optimization

The Brute Force approach involves testing every possible subsequence in the dataset as a candidate s. For each candidate, it computes the minimum distance to every time series in D and evaluates the best split point d to maximize gain.

Number of Candidates: $O(nm^2)$, where n is the number of time series and m is the length of the longest time series. **Cost per Candidate (Split):** The distance computation for one candidate against the whole dataset is $O(n \cdot m \cdot \text{len}(s))$. In the worst case, this is $O(nm^2)$. **Total Complexity:** $O(nm^2) \times O(nm^2) = O(n^2m^4)$.

State-of-the-art optimizations, such as distance caching and the **Early Abandon Distance** (Algorithm), provide significant practical speedups by stopping distance computation early if the accumulated squared distance already exceeds the current best distance found for that time series.

The worst-case complexity remains high, though average complexity improves to approximately $O(n^2m^3)$. The $O(n^2m^4)$ wall remains the primary limitation.

2.3 The FastShapelet Algorithm

FastShapelet is a two-phase approach that overcomes the scalability issue by replacing the expensive brute-force search with a fast, approximate filtering mechanism.

2.3.1 Phase 1: Approximate Filtering (SAX and Random Projection)

1. SAX Mapping: Each real-valued subsequence is first converted into a compact, discrete representation called a SAX word (Symbolic Aggregate approXimation). This reduces dimensionality and enables efficient hashing. **2. Random Projection Hashing:** To make the process robust to minor perturbations, r independent random masks M are applied to the SAX words. A mask defines a subset of SAX positions to retain, generating a *projected signature* $\text{sig}_M(s)$. **3. Distinguishing Power (DP) Score:** The algorithm counts collisions: how many times a SAX word w produces the same signature as other time series from the same class ($A_j(w)$) versus other classes. The $\text{DP}(w)$ score is computed:

$$\text{DP}(w) = 2\sum_{j=1}^{C} A_j(w) - rn$$

The top-k SAX words, ranked by this score, are selected as the only candidates for the final, expensive evaluation. This step drastically prunes the $O(nm^2)$ candidates to just k.

2.3.2 Phase 2: Exact Evaluation in the Original Time Domain

The approximate filtering stage outputs a small set of k highly discriminative *SAX words*. However, SAX words are symbolic abstractions and cannot be used directly to compute exact distances or information gain. Therefore, each selected SAX word must be *remapped* to a real-valued subsequence before the final evaluation.

During the SAX construction phase, the algorithm stores the correspondence between each SAX word and the original time series subsequence from which it was generated, including the index of the time series and the starting position of the sliding window. Using this mapping, every selected SAX word is associated with at least one representative real-valued subsequence of length l. These subsequences form the candidate shapelet set, denoted as TSCand.

For each candidate shapelet $s \in \text{TSCand}$, an exact nearest-neighbor distance is computed with respect to every time series T_i in the dataset:

$$\text{dist}(s, T_i) = \min_j \|s - T_i[j : j + l]\|_2 ,$$

where the minimum is taken over all contiguous subsequences of T_i with length l. This operation yields a distance vector

$$\mathbf{D}(s) = \big[\text{dist}(s, T_1), \ldots, \text{dist}(s, T_n)\big],$$

commonly referred to as the distance list (`DList`).

Using this distance list, all possible split thresholds are evaluated, and the corresponding Information Gain and Gap criteria are computed. The candidate shapelet that maximizes Information Gain (with Gap as a tie-breaking criterion) is selected as the optimal shapelet for the current length l.

This two-stage strategy ensures that approximation is used only to reduce the candidate space, while the final shapelet selection is performed using exact distance computations in the original time domain.

The complexity is now dominated by the $O(r \cdot nm)$ projection step and the $O(k \cdot nm^2)$ exact distance step, making it much more scalable, particularly when $k \ll nm^2$.

3 Data

Our experimental validation primarily utilized the StarlightCurves dataset, a collection of astronomical time series previously employed in related research [3]. This dataset contains light curves, which are measurements of a star's brightness over time, originally gathered by the MACHO (MAssive Compact Halo Objects) project.

The dataset comprises 9,236 distinct light curves. Each curve is characterized by a high dimensionality, containing 1,024 sequential data points. The classification task involves assigning each light curve to one of three categories corresponding to three types of star namely : Eclipsed Binaries (EB), with 2,580 objects, Cepheids (Cep), with 1,329 objects, and RR Lyrae Variables (RR), with 5,236 objects. The dataset is divided into training data and test data of size 1,000 and 8,236 objects, respectively.

Working with StarlightCurves presents several inherent classification challenges that shape the effectiveness of our approach:

- Imbalance: The dataset exhibits class imbalance, with the non-periodic class being significantly overrepresented compared to the other two.
- High Dimensionality: The 1,024 data points per curve result in a large feature space, which often complicates standard classification algorithms.
- Noise Sensitivity: The photometric measurements are inherently noisy, containing brightness fluctuations that are extrinsic to the star's intrinsic class behavior.
- Pattern Ambiguity: Distinct star classes can sometimes generate light curves with superficially similar shapes, making fine-grained discrimination based purely on pattern matching difficult.

4 Results

We note that our article compares its results with the state-of-the-art algorithm, which is too computationally expensive to run. For this reason, we use different methods to compare the shapelets approach with other optimized algorithms, such as the barycenter algorithm (from the reference article [1]) and a Deep Learning (DL) algorithm (mimicking the shapelet discovery process).

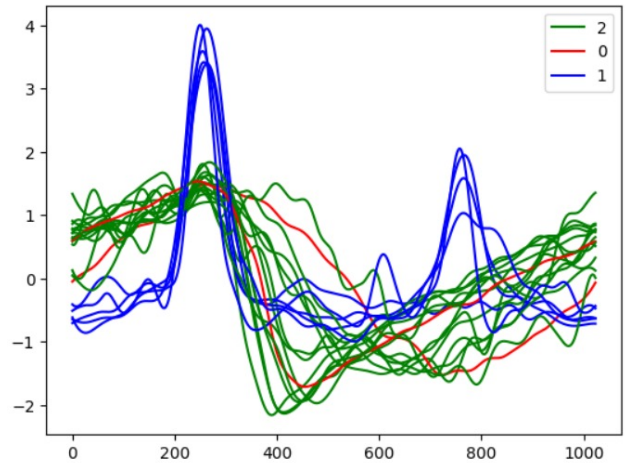

Figure 1: Representative samples from the StarlightCurves dataset for the three classes (Cep (0), Eclipsed Binary(1)) and RR (2))

4.1 The shapelet model performs as well as a DL algorithm on small dasatets (CBF dataset)

Table 1: Comparison of Time Series Classification Methods

Metric	Barycenter Method	DL Methods	Shapelets
Time (Discovery/Training)	120 min	5s	40s
Accuracy (%)	0.97	0.78	0.97

For the Barycenter Method we have taken $\gamma = 1$.

For the DL methods we have not done cross validation, and cut the signal in 4 shapelets.

For the Shapelets method we have taken 3 shapelets of lentgh 20, 60 and 100, the cardinality=4, dimensionality=16,r=10

we recall that: the dimensionality (number of letters per word), the cardinality (alphabet size), and r (the number of random projections).

4.2 The shapelet model performs a slightly better than a DL algorithm on bigger dasatets

Table 2: Comparison of Time Series Classification Methods

Metric	Barycenter Method	DL Methods	Shapelets
Time (Discovery/Training)	∞	1min 30	20min
Accuracy (%)	...	0.84	0.91

Barycenter did not compute (it took too much time)

For the DL method, we used a cross validation split the dataset into 8 segments, a stride=20, we cut the signal into 8 shapelets, number of epochs=16.

For FastShapelets, the length of the shapelet was 100, 200, 300; cardinality=3, dimensionality=16, r=5.

4.3 Hyperparameters :

For the barycenter model, we can tune γ by setting $\gamma =$ median.

For the DL model, we must choose the length of the shapelets considered, the stride between the shapelets, and the number of k windows for cross-validation.

For the FastShapelet algorithm, the key parameters are the lengths of the shapelets, the dimensionality (number of letters per word), the cardinality (alphabet size), and r (the number of random projections).

A Annex: Detailed Algorithms

A.1 Early Abandon Distance Algorithm

Algorithm 1 EarlyAbandonDistance

```
procedure EARLYABANDONDISTANCE(S, S', threshold)
    sum ← 0
    for i = 1 to len(S) do
        sum ← sum + (s_i − s'_i)^2
        if sum ≥ threshold then
            return ∞
        end if
    end for
    return √(sum/len(S))
end procedure
```

A.2 FastShapelet Algorithm (Simplified)

Algorithm 2 FastShapelet (Simplified)

```
Require: D: dataset, r: # projections, k: # candidates
Ensure: best discovered shapelet
[TS, Label] ← ReadData(D)
Global_Shapelet ← null
for len = 1 to m do
    SAXList ← CreateSAXList(TS, len)
    Score ← {}
    for i = 1 to r do
        Count ← RandProjection(SAXList, TS)          ▷ Collision counting
        Score ← UpdateScore(Score, Count)
    end for
    SAXCand ← FindTopK_SAX(SAXList, Score, k, r)
    TSCand ← Remap(SAXCand, TS)                      ▷ Filter candidates to k
    for each cand in TSCand do
        DList ← NearestNeighbor(TS, cand)            ▷ Exact distance computation
        [gain, gap] ← CalInfoGain(DList)
        Update Global_Shapelet if gain/gap is better
    end for
end for
return Global_Shapelet
```

References

[1] Cuturi, M., & Blondel, M. (2017). Soft-DTW: A Differentiable Loss Function for Time-Series. In *International Conference on Machine Learning (ICML)* (pp. 894-903). PMLR.